COMPUTER NETWORKS PROJECT, TEAM 5

Adaptive TCP Load Balancer

SDN programmable data plane with OpenFlow 1.3 and low-level Python sockets

Thanas R PES1UG25AM427

T N Pranav PES1UG25AM421

Sushveen Kaur PES1UG25AM416

PES UNIVERSITY

Department of Computer Science and Engineering (AI&ML)

OVERVIEW

A load balancer that reads the network

1

The problem

Round-robin ignores backend state, so slow or dead servers keep receiving the same share of traffic.

2

The approach

An OpenFlow 1.3 controller polls live telemetry from every backend and scores each one.

3

The outcome

Each new flow goes to the best backend. Failed nodes are isolated and recovered nodes are re-warmed.

Built with Mininet, Open vSwitch, a Ryu controller and plain Python TCP sockets.

SYSTEM FEATURES

What the SRS requires

REQ-1 TO REQ-5

Dynamic Layer 4 balancing

Every new TCP flow is intercepted at the switch and sent to the lowest-scoring backend through the VIP 10.0.0.100:8000.

REQ-6 TO REQ-10

Telemetry and health probing

Each backend answers a JSON poll on its own port every 5 seconds with active connections, latency and error rate.

REQ-11 TO REQ-15

Degradation and failure handling

Three failed polls mark a backend unhealthy and remove it. If every backend is down, new connections are rejected.

REQ-16 TO REQ-19

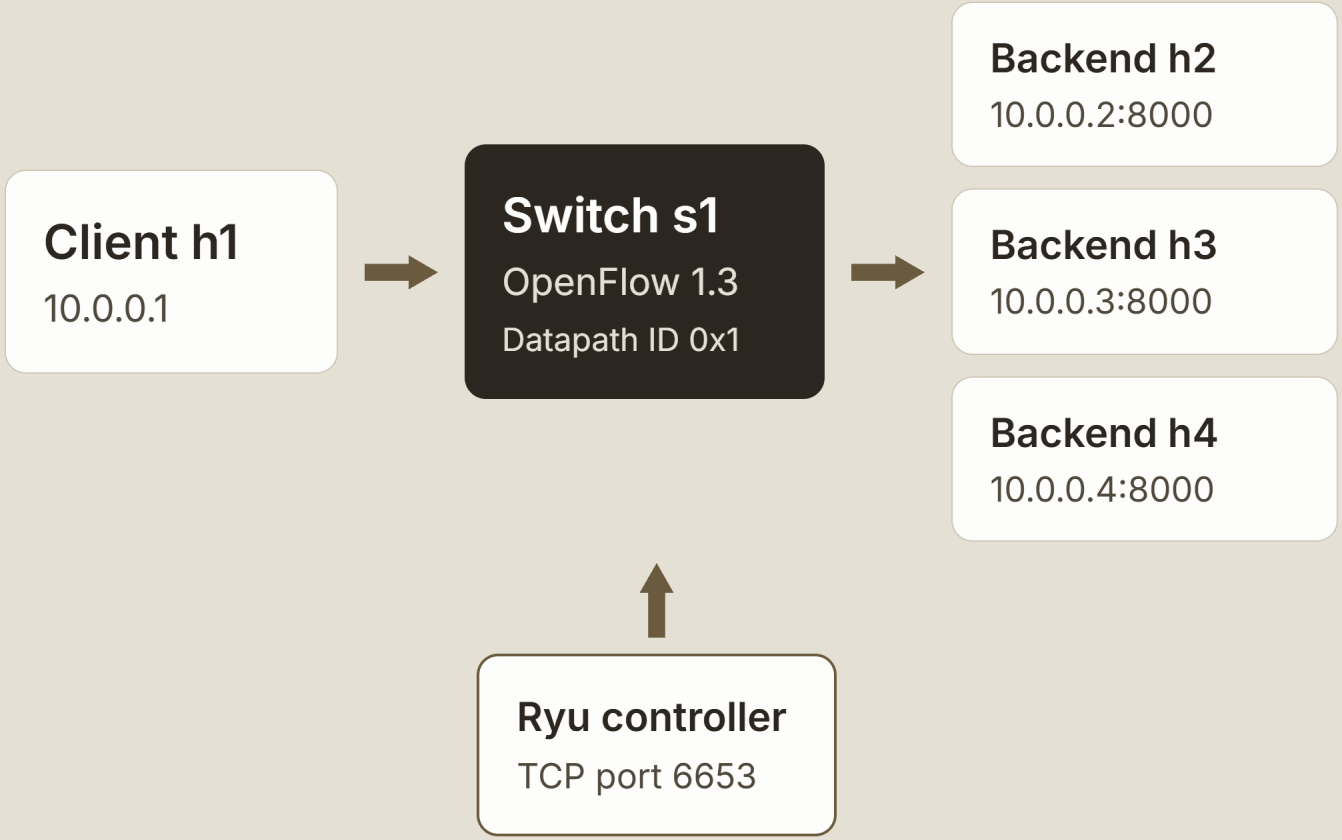
Recovery and re-warming

A recovered backend carries reduced traffic for a 5 second cooldown before it is fully healthy again.

Benchmarking (REQ-20 to REQ-24): light, medium and heavy load with 5, 20 and 50 clients, compared with static round-robin.

ARCHITECTURE

Mininet SDN topology



VIRTUAL SERVICE

10.0.0.100:8000

One entry point and virtual MAC for every client. The backends stay hidden.

SOCKET PROGRAMS

A client worker, a multi-threaded server on port 8000 and a telemetry daemon on ports 9002 to 9004.

HOW A REQUEST FLOWS

From first packet to wire speed

1 **ARP interception**
The controller answers ARP for the VIP with the virtual MAC.

2 **TCP SYN table miss**
The first packet reaches the controller as a PacketIn.

3 **Adaptive assignment**
The scoring engine picks the best backend server.

4 **Bidirectional Flow-Mod**
Two rules are installed, one for each direction.

5 **Wire speed forwarding**
Later packets in the stream bypass the controller.

THE TWO RULES

Client to server

Match client IP and port. Rewrite the destination IP and MAC to the chosen backend. Output to its port.

Server to client

Match backend IP and port. Rewrite the source IP and MAC to the VIP and virtual MAC. Output to port 1.

ADAPTIVE SCORING

Scoring and backend health

0.50	0.35	0.15
Active connections	Smoothed latency	Error rate

Each metric is min-max normalized. Latency uses an EMA (0.25 × new + 0.75 × old) so spikes do not cause route thrashing. The healthy backend with the lowest score gets the next flow.

Healthy

Fully eligible for new flows.

Degraded

Error rate 0.25 or more, or latency 500 ms or more. Score is penalized.

Unhealthy

Three telemetry failures in a row. Excluded from new flows.

Recovering

First success after failure. Penalized for 5 seconds before returning.

MININET DEMO

Running the live demo

1. Terminal 1: start the controller

```
python3 -m controller.ryu_controller
```

2. Terminal 2: start the topology as root

```
sudo python3 -m topology.mininet_topo
```

3. Mininet CLI: send traffic from h1

```
mininet> h1 python3 -m client.load_generator --vip 10.0.0.100
--port 8000 --requests 100 --concurrency 10
```

WHAT TO SHOW

Start the controller first so the switch connects on startup.

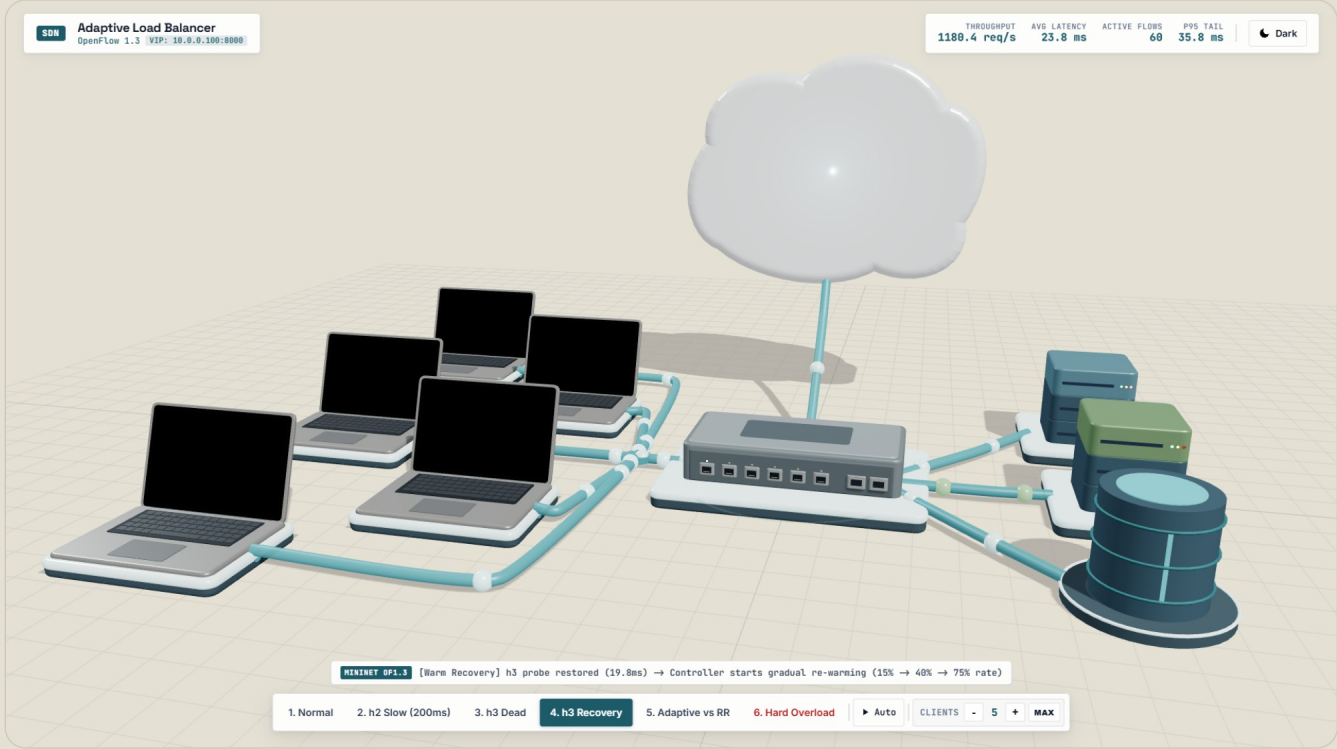
While h1 sends requests, show the controller installing and changing flow rules live.

No Mininet? Run python -m scenarios.run_all_scenarios for all six scenarios on the standalone testbed.

Needs Ubuntu 20.04 or later, Mininet 2.3+, Open vSwitch 2.13+, Ryu 4.34+ and Python 3.8+.

3D ANIMATION DEMO

The network, in motion



Open it

Open visualization/animation.html in any browser. It is built with Three.js.

What you see

Client laptops, an Open vSwitch, backend servers and a floating controller cloud.

Scenario buttons

Normal, h2 slow, h3 dead, h3 recovery, adaptive versus round-robin and hard overload.

Live view

Packets reroute as backend health changes. A bar tracks throughput, latency and flows.

RESULTS

Adaptive versus round-robin

NORMAL LOAD

+10.5%

Throughput rises from 285 to 315 req/s

H2 SLOWED TO 200 MS

−63.2%

Average latency falls from 84.8 ms to 31.2 ms

TRAFFIC TO SLOW H2

8.5%

Round-robin still sends it 33.3%

H3 KILLED MID-RUN

0%

Dropped connections. Round-robin fails 33%

Measured in Mininet with 10 concurrent clients. Six scenarios cover normal load, slowdown, crash, recovery, a benchmark and a hard overload plus kill.